IMAGE PROCESSING PROJECT

Statistical Exploration and Visual Retrieval of Images
Using the MS COCO Dataset

Module: Introduction to Image Processing

Project focus: Image preprocessing, object detection, visual embeddings, similarity search and dataset statistics

Technical Report

Based on the supplied Python notebook and the five statistical visualisations.

Abstract

This project develops an image-analysis and visual-retrieval workflow using the MS COCO 2017 dataset. The notebook prepares separate image splits, corrects image orientation, converts images to RGB, resizes them, cleans their reference captions, extracts object metadata with YOLOv8, and represents images as normalised feature vectors using the OpenCLIP ViT-B-32 model. The vectors are stored in a FAISS flat index to support similarity-based retrieval. Finally, the project explores the dataset through statistics and plots describing object categories, caption length, frequent words, part-of-speech tags and image embedding similarity. The report explains each implemented stage and interprets the supplied figures. It also identifies implementation details that should be checked before treating the index as a final 10,000-image knowledge base.

Report structure

1. Introduction and objectives
2. Dataset and preparation
3. Image-processing workflow
4. Visual feature extraction and retrieval
5. Statistical exploration and results
6. Quality control, limitations and conclusion

1. Introduction

Image processing concerns the transformation and analysis of digital images so that useful information can be extracted from them. In this project, image processing is combined with computer vision and natural-language processing. The image collection is analysed visually through object detection and deep image embeddings, while the associated captions provide a textual description of the scenes.

The project uses COCO because it contains everyday scenes with multiple object categories and human-written captions. This makes it suitable for studying both visual content and the language used to describe images. The supplied project specification frames the broader topic as preparation for comparing retrieval-augmented captioning methods. The notebook covered by this report implements the data-preparation and retrieval foundation; it does not yet implement or evaluate a complete RAG caption-generation system.

1.1 Objectives

• Prepare a reproducible subset of COCO images and their associated captions.

• Apply consistent image transformations before visual feature extraction.

• Detect objects and save their labels, confidence values and bounding boxes as image metadata.

• Generate image embeddings with a pretrained vision-language model.

• Build a FAISS index and demonstrate image-to-image and text-to-image similarity search.

• Explore the dataset statistically and interpret the distributions shown in the supplied figures.

2. Dataset and data preparation

The notebook downloads the COCO 2017 dataset through KaggleHub and locates the train2017 and val2017 image folders together with the caption annotation JSON files. It then creates three lists using a fixed random seed (42): 10,000 images selected from train2017 for the knowledge-base split, 100 different train2017 images for validation, and 1,000 images selected from val2017 for testing. The lists are exported as CSV files so that the selected filenames can be reused.

2.1 Split and leakage control

Split	Source	Number of images	Purpose
Knowledge base	COCO train2017	10,000	Reference collection for retrieval
Validation	COCO train2017, disjoint from knowledge base	100	Configuration/tuning split
Test	COCO val2017	1,000	Held-out evaluation split

The notebook checks that the three filename lists are pairwise disjoint using Python set operations and assertions. The anti-leakage check is therefore a useful control over the selected split lists. The code also extracts the corresponding captions from the COCO annotation files and stores both the original caption and a cleaned version. Original captions are retained so that cleaning does not overwrite the reference text.

2.2 Caption cleaning

The clean_caption function lowercases the text, replaces repeated whitespace with a single space, removes punctuation and trims leading/trailing spaces. The prepare_captions function selects annotations by image ID and creates a table containing image_id, raw_caption and cleaned_caption. Three CSV files are written for the knowledge base, validation and test captions. This separation makes it possible to use cleaned text for indexing or statistics while preserving the original reference captions.

Important implementation note: the specification supplied with the assignment describes a 100-image test subset, whereas this notebook samples 1,000 test images. This report describes the notebook's actual implementation. The final project should confirm which test-set size is required and use it consistently.

3. Image preprocessing

Before feature extraction, the notebook defines a preprocessing function using the Pillow (PIL) library. For each image, the function opens the file, applies EXIF orientation correction, converts the image to RGB and creates a thumbnail constrained to 224 × 224 pixels. The thumbnail operation preserves the image aspect ratio rather than stretching every image to a square. The result is saved as a JPEG in a folder corresponding to its split: knowledge_base, validation or test.

Why these operations matter

• **EXIF correction:** some photographs store rotation information in metadata; applying it makes the pixel orientation consistent.

• **RGB conversion:** standardises colour channels and avoids problems caused by grayscale, palette or other image modes.

• **Resizing:** limits image dimensions and makes later processing more manageable.

• **Split-specific folders:** preserve the separation between knowledge-base, validation and test images.

The notebook later uses the preprocessing transform returned by OpenCLIP when creating embeddings. That transform is distinct from the earlier saved-thumbnail function: it applies the input formatting expected by the

pretrained ViT-B-32 model.

4. Visual analysis and feature extraction

4.1 Object detection with YOLOv8

The notebook loads the lightweight YOLOv8n detector from the Ultralytics package. Each processed detection result is examined to obtain the predicted class ID, class name, confidence score and bounding-box coordinates (x1, y1, x2, y2). These detections are grouped by filename in metadata_dict. For every image, the metadata record stores the filename, the list of detected objects and the total number of detections. The resulting dictionary is intended to be exported as a JSON file.

The detector is called with a confidence threshold of 0.25 and a batch size of 32. The supplied notebook output reports 118,287 images found and shows the metadata extraction progress at 1,031 of 3,697 batches (about 28%). Thus, the captured output demonstrates that extraction started, but does not prove that the full metadata run completed in that execution.

4.2 Image embeddings with OpenCLIP

To represent image content numerically, the notebook loads the pretrained OpenCLIP model ViT-B-32 with the OpenAI weights. The model is placed in evaluation mode and moved to the available device (GPU when CUDA is available, otherwise CPU). For each image, the OpenCLIP preprocessing transform converts it into the tensor format expected by the model. The encode_image method produces a feature vector, which is normalised by dividing it by its L2 norm. The vectors and corresponding filenames are accumulated and saved in NumPy files.

Normalisation is important because it makes the dot product between two feature vectors equivalent to cosine similarity. The notebook uses torch.no_grad() to avoid computing gradients during inference, reducing memory use and computation.

4.3 Building the FAISS similarity index

The saved embedding matrix is loaded and converted to float32, which is the expected numeric type for the FAISS index used here. The code creates faiss.IndexFlatIP with the embedding dimension and adds all vectors to the index. IndexFlatIP performs an exact flat search using inner product. Since the image embeddings have been normalised, the returned inner-product scores can be interpreted as cosine similarities. The index, embedding matrix and filename mapping are saved for later reuse.

A separate knowledge-base metadata table links each vector ID to its filename and image ID, and attaches the captions and YOLO metadata. The notebook also writes a JSON version of this information. The search function encodes a text query using OpenCLIP's text encoder, normalises the text vector and asks FAISS for the top-k closest image vectors. By default, top_k is 5.

4.4 Retrieval sanity check

The notebook searches the index using the first image embedding as its query and requests five results. It asserts that the first returned index is the same vector used as the query. This verifies that the index can retrieve an identical vector at the top of the ranking. It is a basic technical sanity check, not a measurement of semantic retrieval accuracy or caption quality.

5. Statistical exploration and results

The final part of the notebook analyses the captions, detected objects and image embeddings. The five figures supplied with the request are reproduced below. Their interpretation is descriptive: the charts show distributions and relative frequencies, but they do not by themselves establish model accuracy or causal relationships.

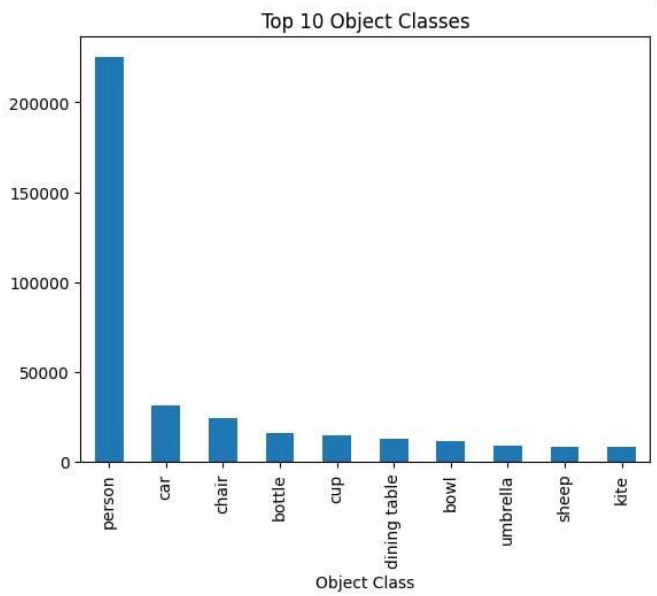


Figure 1. Top 10 detected object classes in the supplied visualisation.

The object-class chart is strongly dominated by the person category. Car and chair are the next most frequent classes, followed by bottle, cup, dining table, bowl, umbrella, sheep and kite. This suggests that people and common everyday objects are prominent in the analysed image collection. The counts are detection frequencies, not necessarily the number of images containing each class: one image can contain several instances of the same class.

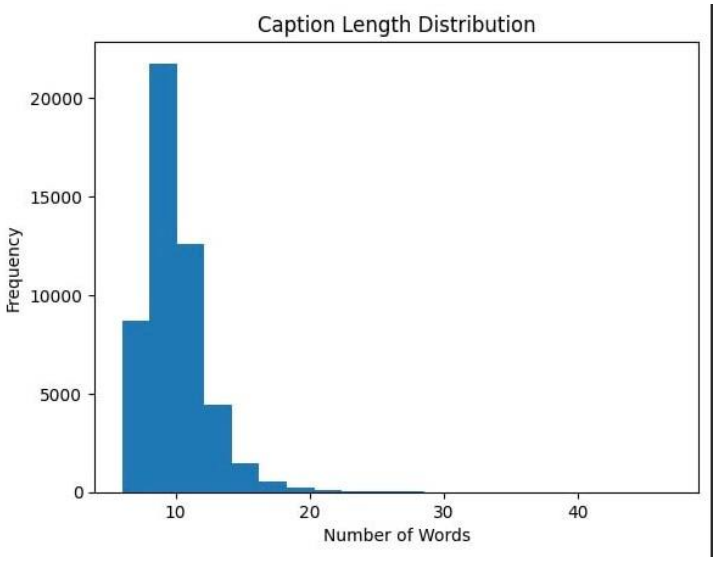


Figure 2. Distribution of caption lengths measured in words.

Caption lengths are concentrated around roughly 8–12 words, with the highest bars near 9–10 words. Frequency decreases as captions become longer, and only a small number of captions extend beyond about 20 words. This indicates that most COCO captions in the sample are concise descriptions rather than long narratives. The notebook calculates caption length by splitting the cleaned caption on whitespace.

5. Statistical exploration and results (continued)

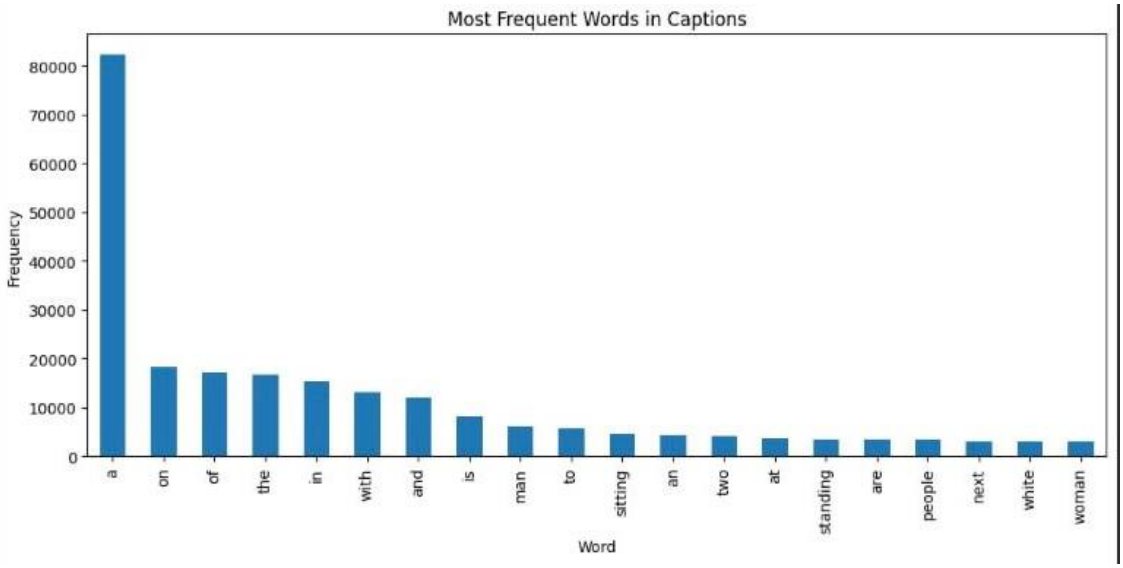


Figure 3. Twenty most frequent words in the cleaned captions.

The most frequent token is “a”, which appears much more often than the other words. Function words such as “on”, “of”, “the”, “in”, “with” and “and” are also common. Content words such as “man”, “sitting”, “standing”, “people”, “white” and “woman” appear among the frequent terms. This pattern is expected in natural-language captions: grammatical words recur across many descriptions, while nouns and verbs describe the visible scene. The code does not remove stop words before producing this chart.

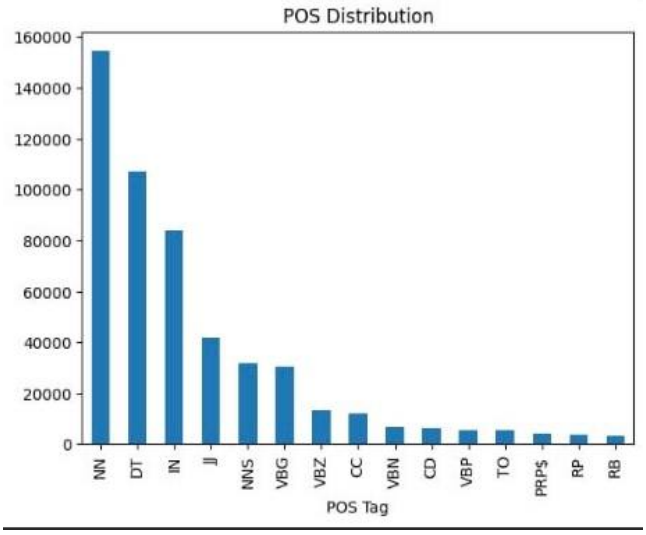


Figure 4. Part-of-speech (POS) tag frequencies in the captions.

Nouns (NN) form the largest POS category, followed by determiners (DT) and prepositions (IN). Adjectives (JJ), plural nouns (NNS) and verb forms such as VBG and VBZ also occur frequently. This distribution is consistent with captions that identify objects and people, then describe their attributes, positions or actions. POS tags are assigned by NLTK's English part-of-speech tagger after tokenisation.

5. Statistical exploration and results (continued)

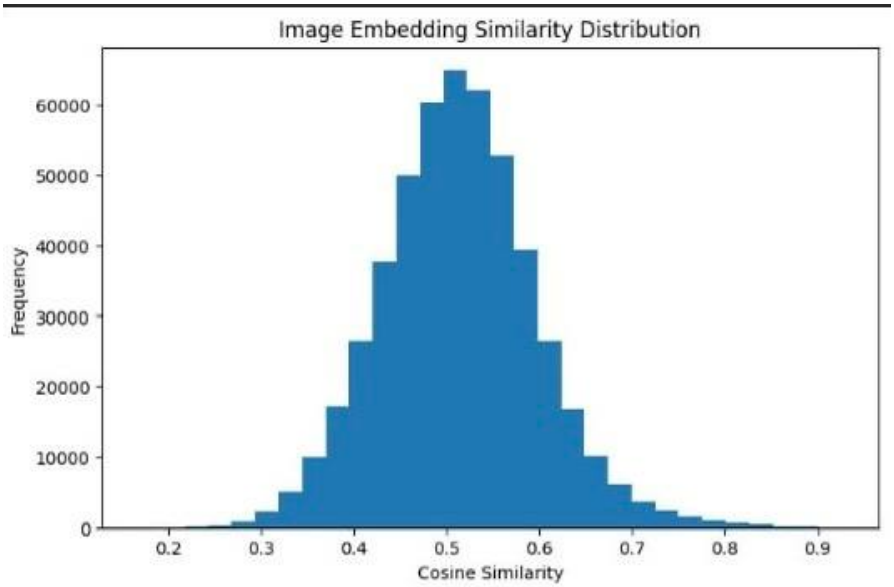


Figure 5. Pairwise cosine similarity distribution for a random sample of image embeddings.

The histogram shows pairwise cosine similarities for a random sample of up to 1,000 image embeddings. The distribution is concentrated approximately between 0.4 and 0.65, with its peak around 0.5–0.55. There are fewer pairs at very low or very high similarity values. This indicates that the image vectors share some common visual-semantic structure while still distinguishing many image pairs. Similarity is not a direct measure of whether two images depict exactly the same objects; it is a score in the learned embedding space.

5.1 Caption vocabulary and diversity

The notebook also calculates lexical diversity as the number of unique words divided by the total number of word tokens. A higher value means that a larger proportion of the tokens are distinct, while a lower value indicates more repetition. Semantic diversity is estimated as one minus the mean pairwise cosine similarity among a random sample of image embeddings. These are useful summary indicators, but their values depend on the dataset size, preprocessing choices and sampling procedure. The supplied figures do not display the exact printed values, so this report does not invent numerical results for either measure.

5.2 Overall interpretation

Taken together, the figures show a dataset containing frequent human-centred scenes and common household or outdoor objects, with short and formulaic captions. The language statistics show a high frequency of grammatical words and nouns, while the embedding histogram suggests that many image pairs have moderate similarity in the CLIP feature space. These observations help describe the data that a later retrieval and captioning system would use.

6. Quality control and implementation considerations

6.1 Checks included in the notebook

• Checks for NaN and infinite values in the embedding matrix.

• Counts duplicate filenames and duplicate image IDs.

• Counts missing metadata and missing caption mappings in the knowledge-base table.

• Checks that the number of embeddings matches the number of stored filenames.

• Checks the overlap between knowledge-base, validation and test split IDs.

• Verifies that the first image embedding retrieves itself at rank one.

6.2 Points to verify before final submission

1. Embedding/index scope: the split creation selects 10,000 knowledge-base images, but the YOLO and OpenCLIP loops use the full train2017 directory. The notebook output reports 118,287 training images. Consequently, unless the input folder was restricted outside the shown code, the metadata and embedding runs operate on the full training set rather than only the 10,000 selected filenames. The FAISS index therefore appears to be built from all saved embeddings, while the caption mapping is built from the 10,000-image knowledge-base captions. This mismatch should be resolved by filtering the image list to knowledge_images before detection and embedding, or by deliberately preparing captions and splits for the full indexed collection.

2. Metadata run completion: the captured YOLO output is at approximately 28% progress. Confirm that the run finishes and that the JSON file contains records for the intended images before loading it later.

3. Metadata coverage: the YOLO metadata extraction loop shown uses the raw train2017 directory and does not itself restrict the operation to the selected knowledge-base split. The report's object chart should be labelled with the exact population used when the notebook is rerun and completed.

4. Retrieval validation: retrieving the query image itself is expected for an exact self-query. For a stronger evaluation, exclude the query image from results and compare retrieved neighbours against human relevance judgements or COCO annotations.

5. Reproducibility: record the exact OpenCLIP, YOLO, FAISS, PyTorch and dataset versions, the number of successfully processed images, embedding dimensions, indexing time and final index size. These values are printed or can be captured by the notebook, but are not visible in the supplied five chart images.

6. Dataset split specification: reconcile the notebook's 1,000-image test set with the assignment document's 100-image test subset before reporting final evaluation results.

7. Conclusion

This project establishes a practical image-processing and visual-retrieval pipeline over MS COCO 2017. It begins with dataset selection and split control, prepares images and captions, extracts object-level metadata with YOLOv8, and creates semantic image representations using OpenCLIP ViT-B-32. A FAISS flat index then makes it possible to retrieve visually similar images and to search images using text queries encoded in the same multimodal embedding space. Statistical analysis provides an overview of the dataset's objects, caption lengths, vocabulary, grammatical structure and embedding similarities.

The five visualisations provide useful descriptive evidence about the dataset, but they are not an evaluation of generated captions or of a RAG method. The most important next step is to align the indexed images, captions and metadata with the selected knowledge-base split, complete and validate the metadata extraction, and then perform retrieval and caption-quality experiments on the held-out test set. With these checks, the implemented work can serve as a reliable preprocessing and retrieval foundation for the wider image-captioning study.

References

1. Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. European Conference on Computer Vision (ECCV).
2. Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML.
3. Ultralytics. YOLO documentation and model resources. https://docs.ultralytics.com/
4. Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
FAISS documentation: https://faiss.ai/
5. OpenCLIP project documentation and implementation: https://github.com/mlfoundations/open_clip
6. NLTK documentation: https://www.nltk.org/